

Multi-Agent Research Assistant

Capstone Project — Plan, Architecture & Work Division

Part 1 — The Project in Plain Language

This section needs no programming background. It is the source material for the presentation.

The problem

A literature review means reading dozens of papers and pulling out what each one actually found. Done by hand it takes weeks.

The obvious shortcut is to hand all the papers to an AI model at once and ask for a summary. This fails in three specific ways, and the failures are hard to spot:

Failure	What it looks like
Blurring	Details from ten papers merge into one vague statement that no single paper actually supports
Misattribution	A finding from Paper C gets cited to Paper A
Fabrication	The model states a number or result that appears in none of the sources

All three produce text that *reads* correct. For a thesis review or a due-diligence report, that is worse than useless — it is confidently wrong, and checking it by hand costs as much as writing it by hand.

The solution

Instead of one model doing everything in one pass, we use **five specialist agents in a pipeline**. Each does one job and hands its output to the next.

#	Agent	Job
1	Query Planner	Breaks the research question into 3–6 focused sub-questions
2	Retriever	Searches academic databases and collects the actual papers
3	Summarizer	Reads one paper at a time and extracts its findings
4	Critic	Checks every extracted finding back against the original paper
5	Synthesis	Writes the final report using only findings that passed the Critic

The key idea (this is what makes the project novel)

Most systems of this kind verify at the *retrieval* level — they check that a relevant paper was found. **We verify at the claim level** — every individual sentence that reaches the final report is checked against the exact source text that supports it.

Here is how. The Summarizer is not allowed to write prose. It must output each finding as a **claim paired with a verbatim quote** copied from the paper:

Claim: Dexamethasone reduced 28-day mortality in ventilated COVID-19 patients.

Quote: *"In the dexamethasone group, 28-day mortality among patients receiving invasive mechanical ventilation was 29.3%, as compared with 41.4% in the usual care group."*

Because the quote must be word-for-word from the paper, we can check it **twice**:

Stage 1 — Does the quote actually exist in the paper? This is a plain text search. No AI involved, no cost, instant. If the model invented the quote, it is caught here and thrown out immediately.

Stage 2 — Does the quote actually support the claim? Now an AI Critic looks at just the claim and just the quote — nothing else, not the other papers. It decides: SUPPORTED, PARTIALLY SUPPORTED, UNSUPPORTED, or CONTRADICTED.

Only claims that survive both stages are allowed into the final report.

Why this design is genuinely better, not just more steps:

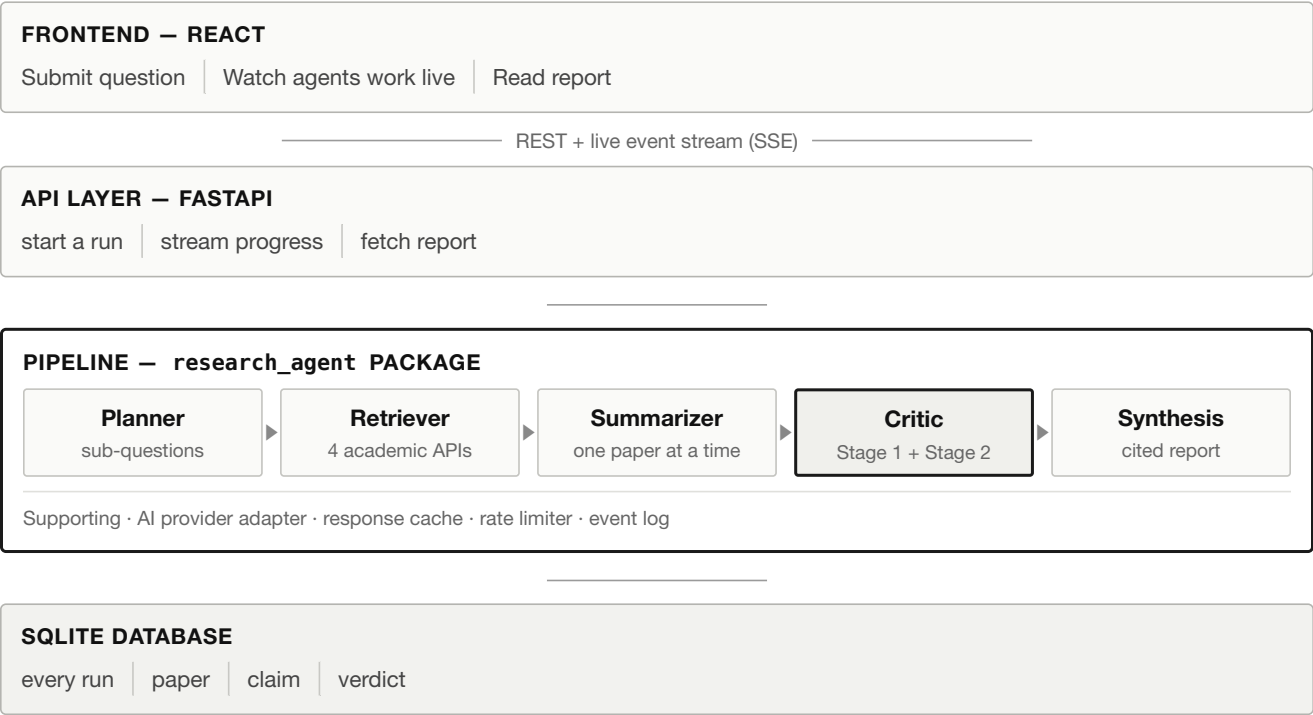
- Stage 1 is free, so we spend our limited AI budget only on claims worth judging.
- The Summarizer sees one paper at a time, so findings *cannot* blur across papers.
- The Critic sees one quote at a time, so it cannot accidentally cross-attribute.
- Every rejection is recorded, so we can **measure** how much hallucination we caught — which becomes the evaluation study.

Glossary

Term	Meaning
Agent	One AI component with a single narrow job
Claim	One atomic factual statement extracted from a paper
Quote / span	The exact sentence from the paper that backs a claim
Hallucination	Something the AI stated that the source never said
Entailment	Whether a piece of text logically supports a statement
Baseline	The naive version (one AI, one pass) we compare our system against
Pipeline	The five agents running in sequence

Part 2 — Architecture

System diagram



The one rule that holds this together: the pipeline package never imports the web server and does not know a website exists. The API and the evaluation harness are two independent users of the same package.

This matters practically — it is why three people can work at once without colliding, and why the evaluation study can run the pipeline directly with no browser involved.

Technology choices

Layer	Choice	Why
Backend	Python 3.11 + FastAPI	Async support for running agents in parallel; built-in live streaming
Frontend	React + Vite + TypeScript + Tailwind	Best fit for the side-by-side claim ↔ evidence view
Database	SQLite via SQLAlchemy	Zero setup, one file, easy to hand in
AI model	Gemini free tier (Groq as backup)	No cost; swappable via one config line
Orchestration	Plain Python <code>asyncio</code>	Written by us, so we can explain it in the viva — no framework black box

On writing our own orchestration instead of using LangGraph: in a capstone defence we have to explain how the system works. "The framework did it" is a weak answer. Our orchestrator is about 150 lines and we can defend every one.

Where the papers come from

Source	Provides	API key
OpenAlex	Primary search, metadata, abstracts	None
arXiv	Free full-text PDFs (CS, physics)	None
Semantic Scholar	Citation counts, TLDRs	Free key
Crossref	DOI resolution, metadata backfill	None

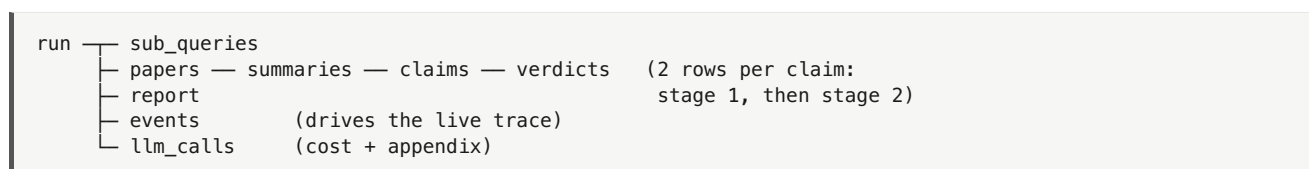
Duplicates are merged by DOI first, then by fuzzy title match. Where a free PDF exists we extract the full text and split it into sections; where it does not, we fall back to the abstract and **mark that paper as abstract-only in the interface**, because a claim drawn from an abstract is weaker evidence.

Working within a free AI tier

Free tiers have daily request caps. Four measures keep this from becoming a problem:

1. **Response cache on disk.** Identical request → instant cached answer, no quota used. Re-running the same question during development costs nothing.
2. **Rate limiter** with automatic backoff when the provider says slow down.
3. **Replay mode.** Any completed run can be replayed from the database with realistic timing and zero API calls. *We record a run the night before each review.* If the network or the quota fails during the demo, the demo still works.
4. **Every AI call is logged** — prompt, response, tokens, timing. This becomes the report appendix and the cost analysis.

Data model



Storing **both** verdicts per claim is deliberate: it lets us report exactly how many hallucinations Stage 1 caught for free versus how many needed the AI Critic — without re-running anything.

Part 3 — Who Builds What

The work divides into three tracks along seams that already exist in the architecture. Each track owns a complete vertical slice of the system with its own deliverables, and after week 1 the tracks are independent: **no track can block another**.

Track A — Runtime & Agents

Everything that runs. The agents that turn a research question into claims, the machinery that sequences them, and the server that exposes the whole thing.

- AI provider adapter, response cache, rate limiter
- Query Planner agent — decomposing a question into focused sub-questions
- Summarizer agent — the claim-plus-verbatim-quote extraction format
- Synthesis agent — theme grouping, citation assignment, BibTeX generation
- All prompt engineering and structured-output schema design
- Orchestrator — async stage sequencing and bounded concurrency
- Live event bus and database schema
- FastAPI server and all endpoints

Track B — Retrieval & Interface

Everything at the system's boundaries — getting real papers in from the outside world, and getting results out to the person using it.

- Four academic source integrations: OpenAlex, arXiv, Semantic Scholar, Crossref
- Deduplication by DOI and fuzzy title matching
- PDF retrieval and full-text extraction with section splitting
- **The complete React frontend** — live agent trace, report view, evidence pane, exports

Track C — Verification & Evaluation

Everything that proves it works. A capstone that claims to reduce hallucination has to *measure* it, and that measurement is a research workstream in its own right — experimental design, a labeled dataset, statistics, and the written argument built on them.

- **Stage 1 span checker** — fabricated-quote detection with fuzzy matching
- **Stage 2 entailment Critic** — prompt design, labels, confidence, repair loop
- Baseline system — the single-pass comparison our approach is measured against
- Experimental protocol: question selection, blind labeling design, annotation guidelines
- The labeling tool and the labeled claim dataset — the study's primary data
- Metrics, the evaluation runner, statistical analysis, and the result charts

Sequencing note. Track A is the root of the dependency graph: both other tracks consume its output, so its part of the week-1 contract matters most. In practice all three tracks develop against the shared fixture file

from week 2, so none of them waits on another — the fixture is what converts a chain of dependencies into three parallel workstreams.

What the three tracks agree on in week 1

Settle these before anyone writes separate code:

1. **models.py** — the shared data shapes (Paper, SubQuery, Claim, Verdict, Report). All code is written against these. Changing one is a conversation, not a commit.
2. **fixtures/sample_run.json** — a complete hand-written example run, written on day 2. Track A builds its agents against it, Track B builds the frontend against it, Track C builds the Critic against it. Nobody waits for anybody.
3. **Claims must always carry their quote and its position.** If this field is ever dropped to "simplify things," Stage 1 verification ceases to exist and the project loses its contribution.

Git: `main` protected, feature branches, every PR reviewed by one other person. That review history is itself evidence of process for the report.

Part 4 — Timeline

Review 1 — working vertical slice

Goal: one research question goes in, a cited report comes out, and we can show the Critic rejecting a real fabricated claim on screen.

Week	Track A — Runtime & Agents	Track B — Retrieval & Interface	Track C — Verification & Eval
1	<i>All three together:</i> repo setup, <code>models.py</code> contract, <code>fixtures/sample_run.json</code> , API keys, git rules		
	Gemini adapter + disk cache returning real responses	OpenAlex returning real papers	Span checker skeleton + first tests on the fixture
2	Planner + Summarizer emitting valid claim-plus-quote JSON	Dedupe; React scaffold reading the fixture	Span checker finished — fuzzy matching, unicode normalization; draft entailment prompt
3	Orchestrator wiring Planner → Summarizer; event bus; database writes; SSE endpoint	React trace page consuming live events	Stage 2 Critic + repair loop, plugged into the orchestrator
4	Synthesis agent — citations + BibTeX	Report page + evidence pane	Fabrication-injection test; replay recording

Mid-week 3 is the checkpoint that matters: the first end-to-end run — question in, claims out, verdicts attached. Everything before it is three people working against a JSON file.

Deliberately cut from Review 1: one search source only, abstracts only (no PDF parsing), sequential rather than parallel summarizing, no evaluation numbers yet.

State these cuts openly and show the Review 2 plan. A narrow slice that works beats a broad one that breaks.

Demo script: submit a question with known contested findings → the trace fills in live, agent by agent → land on the finished cited report → open a rejected claim and show that its quote appears nowhere in the source paper.

Review 2 — complete system + evaluation study

Week	Track A — Runtime & Agents	Track B — Retrieval & Interface	Track C — Verification & Eval
5	Parallel summarization with bounded concurrency; theme clustering	arXiv, Semantic Scholar, Crossref; dedup; PDF full-text extraction	Tune Critic for long full-text quotes, section-aware
6	Prompt refinement from error analysis	UI polish, error states, exports	Baseline system, metrics code, labeling tool
7	<i>All three: run the evaluation</i> — 15–20 questions × 2 systems, 300+ claims labeled blind by all three raters		
8	Prompt freeze; methodology chapter	Final UI pass; replay recording	Statistics, inter-annotator agreement, charts, results chapter

Checkpoints where all three work together

When	What
End of week 1	Contract frozen — <code>models.py</code> and the fixture committed. Nothing else starts first.
Mid week 3	First end-to-end run. If this slips, Review 1 is at risk — treat it as a hard date.
End of week 4	Review 1 rehearsal, using the replay recording rather than a live run.
Week 7	Labeling sprint — blocked-out time, all three rating independently, no other work.
End of week 8	Final rehearsal and report assembly.

Part 5 — The Evaluation Study

This is the graded centerpiece. We compare our pipeline against a **baseline**: one AI model, one pass, all abstracts in a single prompt, asked to summarize with citations — i.e. the obvious shortcut described in Part 1.

Metric	What it measures
Claim support rate	Share of claims in the final report a human confirms is genuinely supported by its cited source
Fabricated quote rate	Share of quotes not present in the source — measured <i>before</i> filtering, to show what Stage 1 removes
Citation accuracy	Whether each claim is attributed to the paper it actually came from (catches misattribution)
Stage attribution	How many rejections came from free Stage 1 vs paid Stage 2 — the cost-efficiency argument
Coverage	Supported claims kept per paper — guards against the system scoring well by rejecting everything
Cost	Tokens and wall-clock time per run vs the baseline

Labeling protocol. Mix pipeline and baseline claims together, strip which system produced each one, and have all three of us label independently. Report **Fleiss' κ** for agreement across three raters.

Blind labeling and a reported agreement score are cheap to do and disproportionately raise how seriously the study is assessed.

Expected result, stated honestly: the pipeline will trade some recall and considerably more tokens for a large gain in claim support rate. Report that trade-off as a finding. A study that reports its costs is more credible than one that reports only wins.

Part 6 — Presentation Outline

A slide-by-slide skeleton for the review deck. Content for each slide is in Part 1 and 2.

#	Slide	Content
1	Title	Project name, three members, guide
2	The problem	Weeks of manual reading — with a real number of papers
3	Why the obvious fix fails	The three failure modes table from Part 1
4	Live example	A real fabricated claim from an early test — the hook
5	Our approach	The five-agent pipeline diagram
6	What each agent does	The five-agent table
7	The key idea	Claim + verbatim quote; why prose is banned
8	Two-stage verification	Stage 1 free text search → Stage 2 AI Critic
9	Why this beats one big prompt	Isolation prevents blurring and misattribution
10	System architecture	The block diagram from Part 2
11	Technology stack	The stack table, with the <i>why</i> column
12	Working within a free AI tier	Cache, rate limiter, replay mode
13	Demo	Live run, then a rejected claim shown in the evidence pane
14	How we measure success	The metrics table
15	Results	Charts (<i>Review 2 only</i>)
16	Work division	The three tracks
17	Limitations & future work	The out-of-scope list, framed as deliberate choices

Presentation advice: slides 7 and 8 are the whole project. Spend real time there. Everything else is context. If the reviewer remembers one thing, it should be "*they check every sentence against the actual source text.*"

Part 7 — Risks & Scope

Risk	Mitigation
Free AI quota runs out during a demo	Response cache + replay mode; record a run before each review
Free tier terms change	All model choices in one config file; backup provider adapter ready
PDF text extraction is messy	Abstract-only fallback, flagged in the UI; fuzzy matching absorbs the noise
One track blocks another	Sample fixture file on day 2 — non-negotiable
Labeling runs out of time	Scale to 150 claims; blind protocol and agreement score matter more than volume
Scope creep	The list below is fixed

Deliberately out of scope — name these in the report as choices, not omissions: vector/embedding search, user accounts, multi-user persistence, a chat interface, model fine-tuning, non-English sources.

Appendix — Repository Layout

```
Capstone/
├── backend/
│   ├── research_agent/           # the pipeline – knows nothing about the web
│   │   ├── models.py            # * shared data contracts
│   │   ├── config.py            # model choices, limits, keys
│   │   ├── db.py                # database tables
│   │   ├── orchestrator.py      # stage sequencing + concurrency
│   │   ├── events.py            # live progress event bus
│   │   ├── llm/                 # provider adapter, cache, rate limiter
│   │   ├── agents/              # planner, retriever, summarizer, critic, synthesizer
│   │   ├── sources/             # openalex, arxiv, semantic_scholar, crossref, dedupe
│   │   ├── verification/        # span_check.py, entailment.py
│   │   └── prompts/             # prompts as files, not buried in code
│   ├── api/                     # FastAPI routes + live stream
│   ├── eval/                    # baseline, metrics, dataset, runner
│   ├── cli.py                   # run the pipeline without any UI
│   └── tests/
├── frontend/src/                # React – pages, components, hooks
├── fixtures/sample_run.json     # * unblocks all three tracks on day 2
└── docs/                        # report drafts, diagrams, slides
```

How to verify it works, at any point

```
# unit tests – the span checker is the critical one
cd backend && pytest tests/ -v

# end-to-end, no UI involved
python -m backend.cli run "Do statins reduce dementia risk?" --max-papers 8
python -m backend.cli replay <run_id>           # cached, zero API calls

# full stack
cd backend && uvicorn api.main:app --reload      # :8000
cd frontend && npm run dev                       # :5173

# the evaluation study
python -m backend.eval.run_eval --systems both
```

Two tests worth writing early, because they double as report material:

1. **Fabrication injection** — take a real summary, corrupt one quote (change a number, swap a drug name), confirm the Critic rejects it. This is the proof the Critic does something.
2. **Cross-paper attribution** — feed two papers with similar topics but opposite findings, confirm no claim from one is ever cited to the other.